4 Methodology

This project is built as a sequential pipeline rather than a single model. Its methodology brings together two machine learning components, a rule-based decision layer, and an evaluation engine. The first component is a supervised deep-learning text classifier that turns each news headline into a sentiment score, and it is paired with an independent commercial sentiment signal that acts as an external benchmark. The second component, planned as a comparison model, is a recurrent neural network that learns a trading signal from sequences of sentiment and price features. A signal-generation layer connects them, converting daily sentiment into discrete trade entries, and a fixed-horizon backtesting engine evaluates those entries across a range of holding periods. The subsections below describe the news data sources, each modeling stage, the data splits and training procedure, and the planned optimization. All performance results are reserved for the Results section.

4.1 News Data Sources

News headlines are drawn from two sources, so that no single vendor limits the history depth or coverage available to the study. The first source is a company-news interface (Finnhub) with clean per-ticker attribution. On its free tier, however, it returns only about the most recent year of articles, which is too shallow for the longer holding horizons. The second source is the Alpha Vantage market-news-and-sentiment interface, a documented public application programming interface whose news archive reaches back several years. For each article it returns a ticker-level relevance score, its own sentiment score, and a set of topic classifications. A third source, the open GDELT archive, is kept as a no-cost fallback that reaches back furthest of all. Its company linkage is noisier, because it matches articles to firms by name rather than by explicit tagging. Drawing on more than one source also lowers the risk that quirks of a single vendor's collection are mistaken for properties of the news itself.

The Alpha Vantage source contributes in two ways. First, it addresses the data-depth problem directly, supplying the multi-year history that the longer horizons need in order to accumulate enough non-overlapping trades to be meaningful. Second, it ships its own article-level sentiment score from an independent commercial model, which gives the project an external benchmark. The same articles can be scored by both systems, so the analysis can report whether the two agree and whether trading signals built on each behave alike. Every incoming article, whatever its source, is normalized to a common record format and passed through the same cleaning and point-in-time alignment steps used elsewhere in data preparation, so the choice of source does not leak future information into the backtest. The commercial score is carried through the pipeline as an extra feature rather than as a substitute for the project's classifier. This keeps the deep-learning text model as the primary method while still enabling the comparison.

4.2 Sentiment Classification Model

The sentiment classifier is a transformer-based model in the BERT family Its encoder stacks twelve transformer layers, each with twelve self-attention heads and a hidden representation size of 768, for a total of roughly 110 million parameters. A three-way classification head sits on the pooled representation of the leading token and produces a probability distribution over negative, neutral, and positive tone through a softmax output. This architecture was chosen because most of its linguistic knowledge is acquired during unsupervised pre-training on large text corpora. As a result, it can be specialized to financial sentiment from only a few thousand labeled examples, an approach shown to work well in the financial domain by Araci (2019).

The model is trained in two modes. In the primary mode, the generic pre-trained BERT weights are fine-tuned directly on the labeled financial-sentiment corpus, which makes a full learning curve observable. In the alternative mode, an already domain-adapted financial checkpoint is fine-tuned further. That domain-adapted checkpoint also doubles as a fallback scorer, so a problem in the training environment cannot halt the rest of the pipeline. Once trained, the classifier is applied to every collected headline, and its three output probabilities are reduced to a single sentiment score defined as the positive probability minus the negative probability, a value between negative one and positive one. This scalar keeps both the direction and the confidence of the sentiment: a confidently neutral article scores near zero, while a strongly polar article scores near the matching extreme. When an article also carries a commercial sentiment score from Alpha Vantage, that value is retained on the same scale, so the two independent measures of tone can be compared directly and each can drive the signal layer in turn.

4.3 Feature Aggregation and Signal Generation

The scored headlines are aggregated into a daily panel with one row per stock per trading day. Alongside a simple average of the day's article scores, the pipeline computes a confidence-weighted average in which each article is weighted by one minus its neutral probability, so that decisive articles shape the daily tone more than ambiguous ones. Short rolling averages over a three-trading-day window smooth single-article noise and capture multi-day news narratives. Every rolling feature uses current and past information only, which preserves the point-in-time discipline set during data preparation and guards against look-ahead bias (López de Prado, 2018). A set of standard price-based features is derived for each stock as well, including short- and medium-term moving averages, one-month momentum, one-month realized volatility, and a relative-strength oscillator.

The decision layer turns this panel into discrete trade entries using a deliberately simple and interpretable rule. The central experimental variable in this study is the holding horizon, not the sophistication of the signal, and a more elaborate signal would risk confounding the horizon comparison with its own quirks. A long entry is triggered for a stock on a given day when three conditions hold, using only information available that day: the rolling sentiment exceeds a positive threshold of 0.35, at least two articles support the reading within the window, and the closing price sits above its twenty-day moving average. That last condition is a trend filter, and it avoids buying favorable news in a name that is already declining. Short entries mirror these conditions with a symmetric negative threshold and can be switched off to run the strategy long-only. The numerical values of these thresholds are treated as tunable hyperparameters, addressed under optimization below.

4.4 Multi-Horizon Backtesting and Baselines

The evaluation engine is the methodological core of the project. It applies one identical stream of trade entries at eight fixed holding horizons, spanning one, three, five, ten, twenty-one, forty-two, sixty-three, and one hundred twenty-six trading days. Holding the entries fixed isolates the horizon as the only variable that changes across comparisons. The engine's execution model builds in several safeguards drawn from the backtesting literature (Bailey, Borwein, López de Prado, & Zhu, 2014; López de Prado, 2018). Trades are filled one trading day after the signal is observed, so no position is opened using information from the same closing bar that generated it. Each round-trip trade is charged a fixed transaction cost of ten basis points per side, which matters because turnover, and therefore cost, is highest at the shortest horizons. A stock cannot be re-entered while a position in it is already open, and the number of positions open at once is capped at ten and equally weighted, which keeps the daily portfolio return series realistic rather than assuming unlimited capital. For each horizon the engine produces both a per-trade record, used to summarize the hit rate and the distribution of trade returns, and a daily portfolio return series, used to compute the Sharpe ratio (Sharpe, 1994) and the maximum drawdown.

Two baselines frame every comparison. The first is a buy-and-hold position in the equal-weighted universe, which shows whether the strategy adds value over simple passive exposure. The second is more demanding: a random-signal null. For each horizon, one hundred strategies are simulated that draw entry dates and stocks at random while keeping the same number of trades, the same horizon, and the same cost model as the sentiment strategy. This produces a full distribution of outcomes that represents no skill combined with identical mechanics. Measuring the sentiment strategy against that distribution ensures ordinary market drift or the structure of transaction costs is not mistaken for genuine signal.

4.5 Training Procedure and Data Splits

The labeled sentiment corpus is divided into training, validation, and test partitions in an eighty, ten, and ten percent split, stratified by class so that each partition reflects the same tone distribution. The classifier is fine-tuned with the cross-entropy loss and the AdamW optimizer at a learning rate of two times ten to the negative fifth, a weight decay of 0.01, and a linear warm-up over the first ten percent of training steps. The small learning rate is standard practice for fine-tuning, since larger rates tend to overwrite the pre-trained weights. Training uses a batch size of thirty-two, a maximum input length of 128 tokens, which comfortably fits news headlines and corpus sentences, and runs for three epochs with a fixed random seed for reproducibility. The checkpoint carried forward is the one that maximizes the macro-averaged F1 score on the validation partition. Macro-averaged F1 is chosen over raw accuracy because the tone classes are imbalanced toward neutral, and because the trading application depends specifically on identifying the minority positive and negative cases. The held-out test partition is scored only once, after selection, to characterize final classification quality.

The trading pipeline separates tuning and evaluation on the same principle, applied chronologically: signal parameters are tuned on an earlier in-sample period and then evaluated once on a later, untouched out-of-sample period, so that no part of the reported comparison is fitted to the evaluation data. How demanding this split can be depends directly on the depth of the news history. With only the initial single-vendor source, the usable history spanned roughly five and a half months. That short window forced both periods to be drawn from within it and left the longest horizons with too few non-overlapping trades to interpret, since a position held for one hundred twenty-six trading days could not even complete inside it. Extending the history with the multi-year source restores a conventional calendar-based split, in which earlier years are used for tuning and a recent year is held out for evaluation, and it lets the longer horizons accumulate enough trades to be meaningful. Where a horizon still has too few completed trades for a stable estimate, it is reported with explicit caution rather than dropped, so the decay pattern can be read across the full range while its least-supported points stay clearly flagged.

4.6 Validation and Planned Optimization

Before any conclusion is drawn from live data, the whole apparatus is validated on synthetic data with a known, planted effect. A synthetic market is generated in which prices follow random walks, except that sentiment impulses nudge the next few days of returns by an amount that decays geometrically. Because the ground truth is fixed by construction, a correct pipeline should recover a decay pattern that is strong at short horizons and fades to nothing at longer ones. This validation confirms that the aggregation, signal, backtest, and baseline machinery behave as intended, so that any pattern later seen in the real data can be attributed to the data rather than to a defect in the code.

Model optimization is the focus of the next stage of the project, and it is described here as procedure only, with comparative results reserved for the Results section. For the signal layer, the tunable elements are the sentiment entry threshold, the length of the rolling aggregation window, and the minimum number of supporting articles. These are explored over a small grid and evaluated only on the tuning portion of the data, to avoid overfitting the evaluation period. For the sentiment classifier, the adjustable elements include the learning rate, the number of training epochs, and the choice between fine-tuning the generic or the domain-adapted checkpoint. A second, planned model will add a recurrent neural network based on long short-term memory units (Hochreiter & Schmidhuber, 1997), which have been shown to work well for financial return prediction (Fischer & Krauss, 2018). It will take fixed-length sequences of the combined sentiment and price features and learn to predict the direction of the forward return, providing a learned alternative to the rule-based signal and allowing the two approaches to be compared on the same horizon grid. Its architectural choices, such as the number of recurrent units and layers, will be finalized during the optimization stage.

Use of AI tools: I used Anthropic's Claude (Anthropic, 2026) to assist with the pipeline code (specifically the code to access the APIs of the various financial tools) and correcting/grammar checking draft of this section, I designed the methodology and set all parameters..

Works Cited (Section 4 additions)

Alpha Vantage. (n.d.). Market news & sentiment API [Data set]. https://www.alphavantage.co/documentation/

Anthropic. (2026). Claude [Large language model]. https://claude.ai

Araci, D. (2019). FinBERT: Financial sentiment analysis with pre-trained language models (arXiv:1908.10063). arXiv. https://arxiv.org/abs/1908.10063

Bailey, D. H., Borwein, J. M., López de Prado, M., & Zhu, Q. J. (2014). Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance. Notices of the American Mathematical Society, 61(5), 458-471.

Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. European Journal of Operational Research, 270(2), 654-669.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780.

López de Prado, M. (2018). Advances in financial machine learning. Wiley.

Sharpe, W. F. (1994). The Sharpe ratio. The Journal of Portfolio Management, 21(1), 49-58.